

MODULE 6 · SCHEDULING, RESOURCES & SCALING

Requests & Limits

What the scheduler reserves vs. the hard ceiling the kubelet enforces

Every container needs a floor and a ceiling

Leave resources unset and two things go wrong: the scheduler has nothing to reserve, so Pods land on already-crowded nodes, and a runaway container can consume everything on its node, starving its neighbours.

requests and **limits** fix both problems — but they're answered by different components, at different times, and exceeding them has very different consequences.

BY THE END YOU CAN

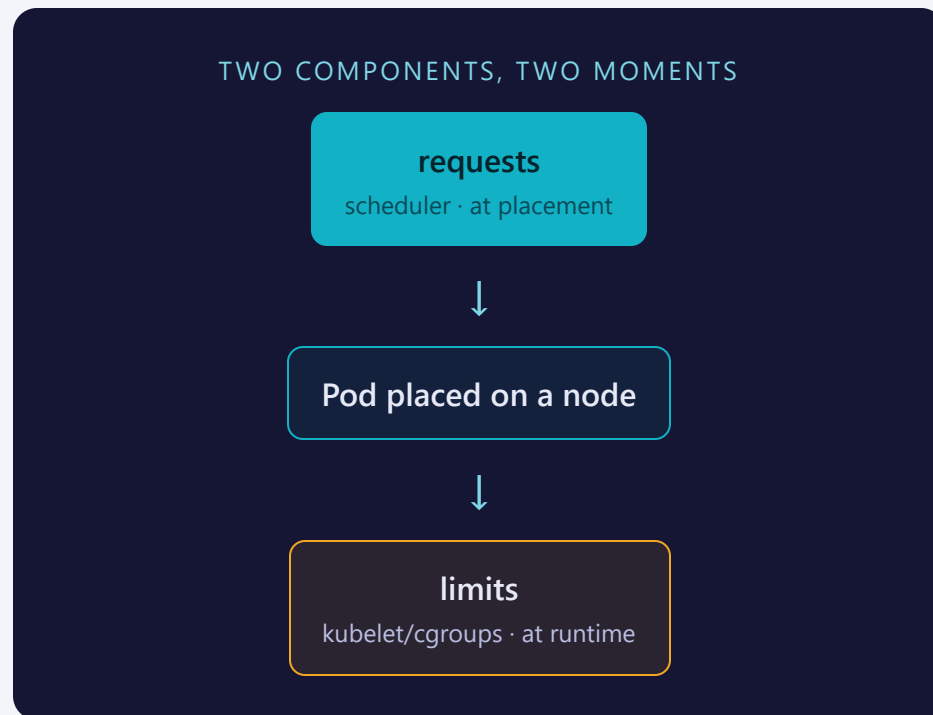
- State what each of requests/limits controls
- Explain why memory over-limit kills, CPU over-limit doesn't
- Read a node's "Allocated resources"
- Say why requests matter even when limits are generous

CONCEPT

Requests reserve, limits cap

- **requests** — read once, by the **scheduler**, when deciding where to place the Pod
- **limits** — enforced continuously, by the **kubelet + container runtime**, via cgroups

Once a Pod is running, nothing checks its requests again — they were only ever a placement-time reservation, not a runtime cap.



Same field, two completely different jobs

	REQUESTS	LIMITS
Checked by	the scheduler	the kubelet + container runtime (cgroups)
Checked when	once, at Pod placement	continuously, while running
Nature	soft floor — an accounting figure	hard ceiling — a cgroup-enforced cap
Exceed it	not possible to "exceed" — it's a reservation	memory → OOMKilled ; CPU → throttled

MENTAL MODEL

Requests answer "will it fit?" Limits answer "how far can it grow?" — asked by different components, at different times.

One block, per container

```
apiVersion: v1
kind: Pod
metadata:
  name: resource-pod
  namespace: training
spec:
  containers:
  - name: nginx
    image: nginx:1.25
    resources:
      requests:           # scheduler reserves this
        cpu: 100m
        memory: 128Mi
      limits:             # hard ceiling, enforced at runtime
        cpu: 200m         # over → throttled
        memory: 256Mi     # over → OOMKilled
```

- **cpu** — measured in millicores; 100m = a tenth of a core
- **memory** — bytes, usually written as Mi/Gi
- Limits should always be $\geq$ requests — the gap is a container's burst room

This is `labs/6.2/pod-resources.yaml` in full.

Imperative to explore, declarative to keep

IMPERATIVE — FAST, ONE-OFF

DECLARATIVE — VERSIONABLE, REPEATABLE

```
kubectl run web --image=nginx --requests=cpu=100m,memory=128Mi --limits=cpu=200m,memory=256Mi
```

```
kubectl apply -f pod-resources.yaml
```

Builds the resources block for you, submits it now

You own the file; it's reviewable and re-appliable

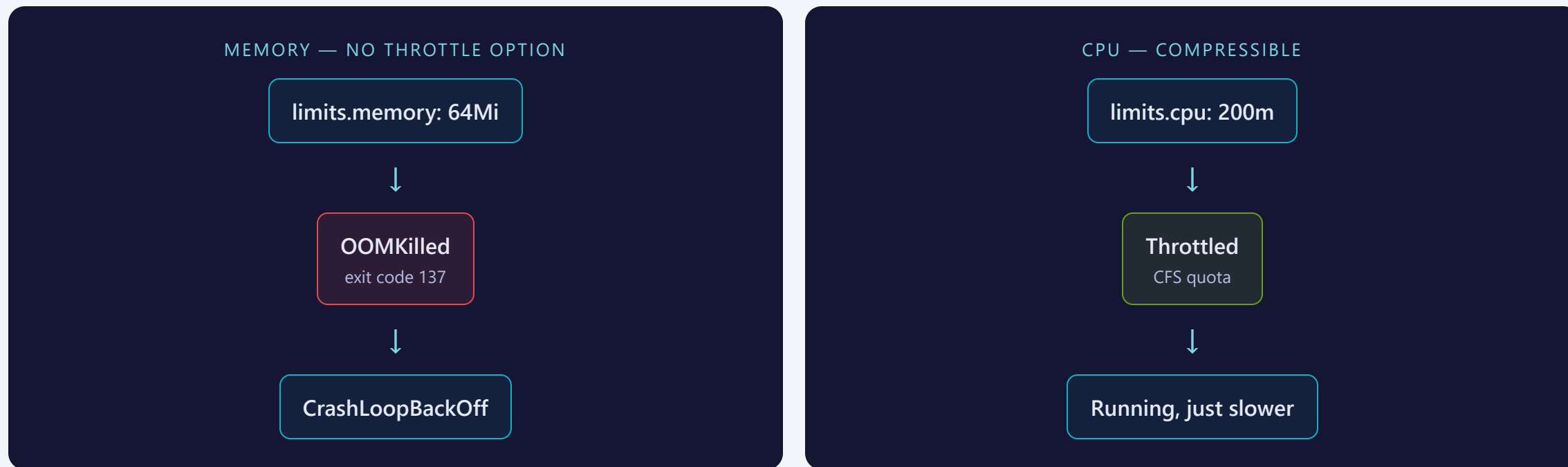
THE BRIDGE TRICK

Generate YAML from an imperative command, then edit & keep it:

```
$ kubectl run web --image=nginx --requests=cpu=100m \  
  --dry-run=client -o yaml > pod.yaml
```

SEE IT

Same overshoot, two different endings

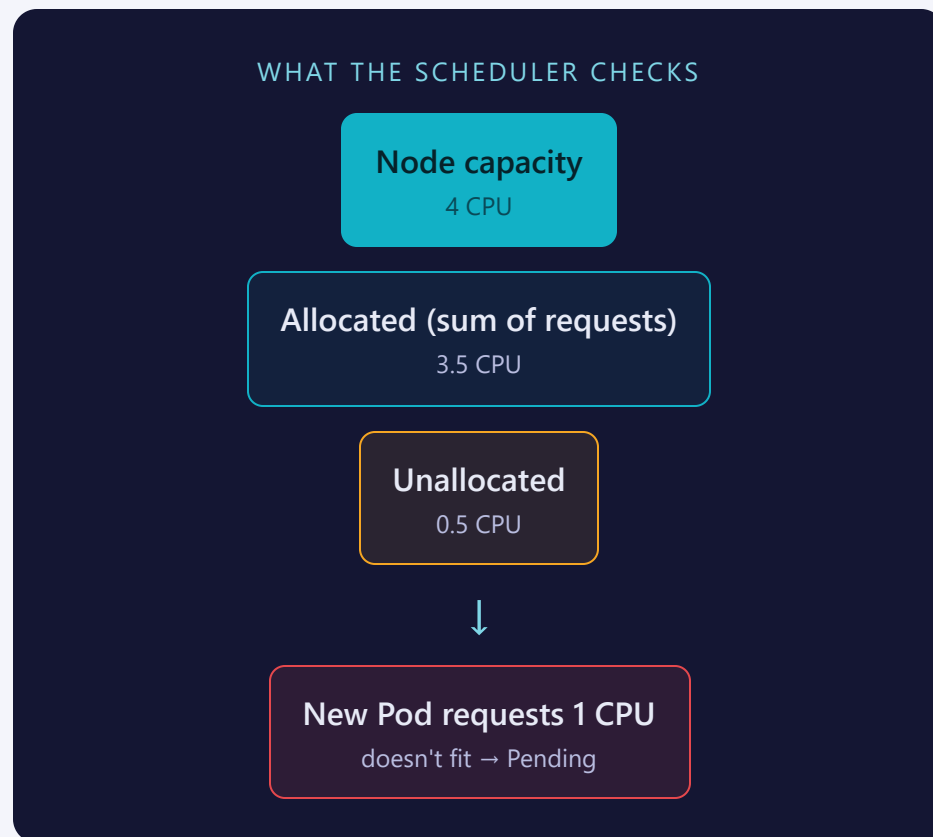


kubectl describe pod mem-limit-demo shows Last State: Terminated, Reason: OOMKilled; kubectl top pod cpu-limit-demo reads capped near 200m while STATUS stays Running.

No requests, no accounting

The scheduler sums the requests already placed on a node and only adds a new Pod if enough **unallocated** capacity remains. `kubectl describe nodes` shows exactly this figure under **Allocated resources** — and it's requests, never the higher limits.

Skip requests and the scheduler assumes the Pod needs almost nothing, happily over-packing the node. Under real load those under-declared Pods are also **first in line** when something has to be evicted.



Where people trip

- **No requests set.** The scheduler assumes almost nothing — Pods land badly and get evicted first under pressure.
- **Expecting a CPU limit to save availability.** It only throttles; it never relieves memory pressure.
- **requests == limits everywhere.** "Safe," but no burst headroom for spiky workloads.
- **Reading kubectl top too soon.** Metrics Server's default scrape interval is 60s — numbers lag reality.

Commands to keep at hand

```
$ kubectl get pod NAME -o jsonpath='{.spec.containers[0].resources}'  
$ kubectl describe pod NAME | grep -A5 "Last State"    # OOMKilled + exit code  
$ kubectl top pod NAME -n NS                          # live usage vs limit  
$ kubectl top nodes  
$ kubectl describe nodes | grep -A5 "Allocated resources" # requests reserved
```

Tip: a CPU-throttled Pod still shows Running — top, not get, is what reveals it.

RECAP

Requests & limits in one breath

- `requests` = the scheduler's soft floor, checked once at placement
- `limits` = the kubelet's hard ceiling, enforced continuously via cgroups
- Memory over limit → OOMKilled; CPU over limit → throttled, keeps running
- No requests → poor placement, and first to be evicted under pressure

HANDS-ON

Now run Lab 6.2
[labs/6.2/](#)

Next: [6.3 — ResourceQuota & LimitRange](#)